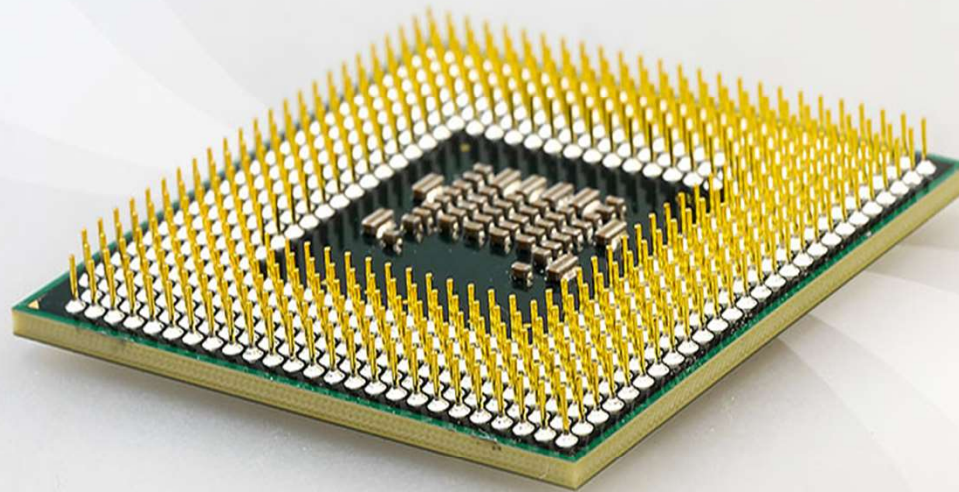


Đại Học Công Nghiệp TP HCM



THIẾT KẾ MẠCH TÍCH HỢP VỚI HDL

GV: Vũ Thị Hồng Nga

Biên soạn năm 2019

Chương 4: Thiết kế mạch tổ hợp với Verilog HDL

Nội dung:

Mô tả các mạch tổ hợp sau

- Mạch cộng (FA, HA)
- Mạch chọn kênh (MUX)
- Mạch phân kênh (DEMUX)
- Mạch giải mã (DECODER)
- Mạch mã hóa (ENCODER)
- Mạch giải mã led bảy đoạn
- Mạch so sánh
- Mạch nhân
- ALU
- Một số IC thông dụng

Các cách mô tả mạch tổ hợp dùng verilog



Mô hình cấu trúc



Mô hình luồng dữ liệu



Mô hình hành vi

I. Mô hình cấu trúc (Structural model)

1. Đặc điểm chung:

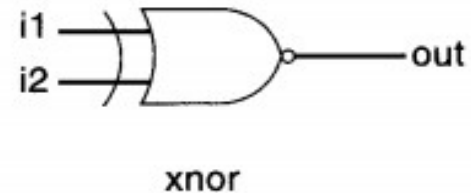
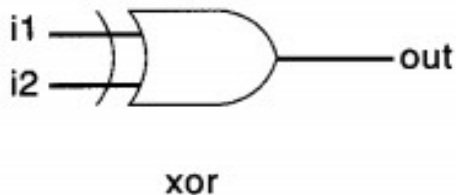
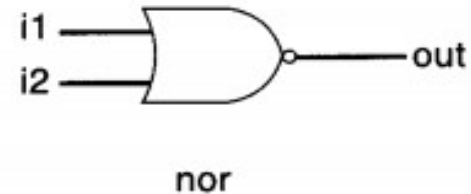
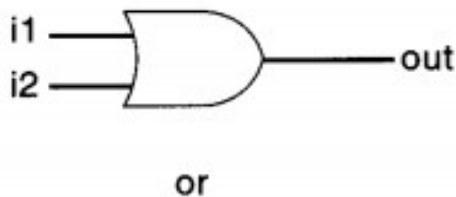
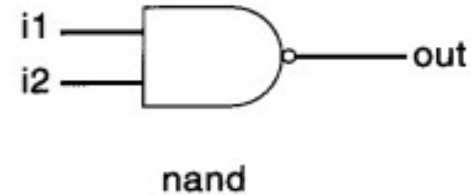
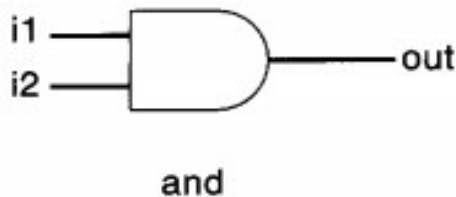
- Mô hình cấu trúc mô tả các hệ thống dưới dạng các cổng logic hay các khối linh kiện được kết nối lại với nhau để thực hiện được những mạch số mong muốn.
- Mô hình cấu trúc được mô tả một cách trực quan thiết kế số vì mạch logic sau khi tổng hợp sẽ giống với mạch số thông thường
- Mô hình cấu trúc thường áp dụng cho những module nhỏ, không cần tối ưu về timing



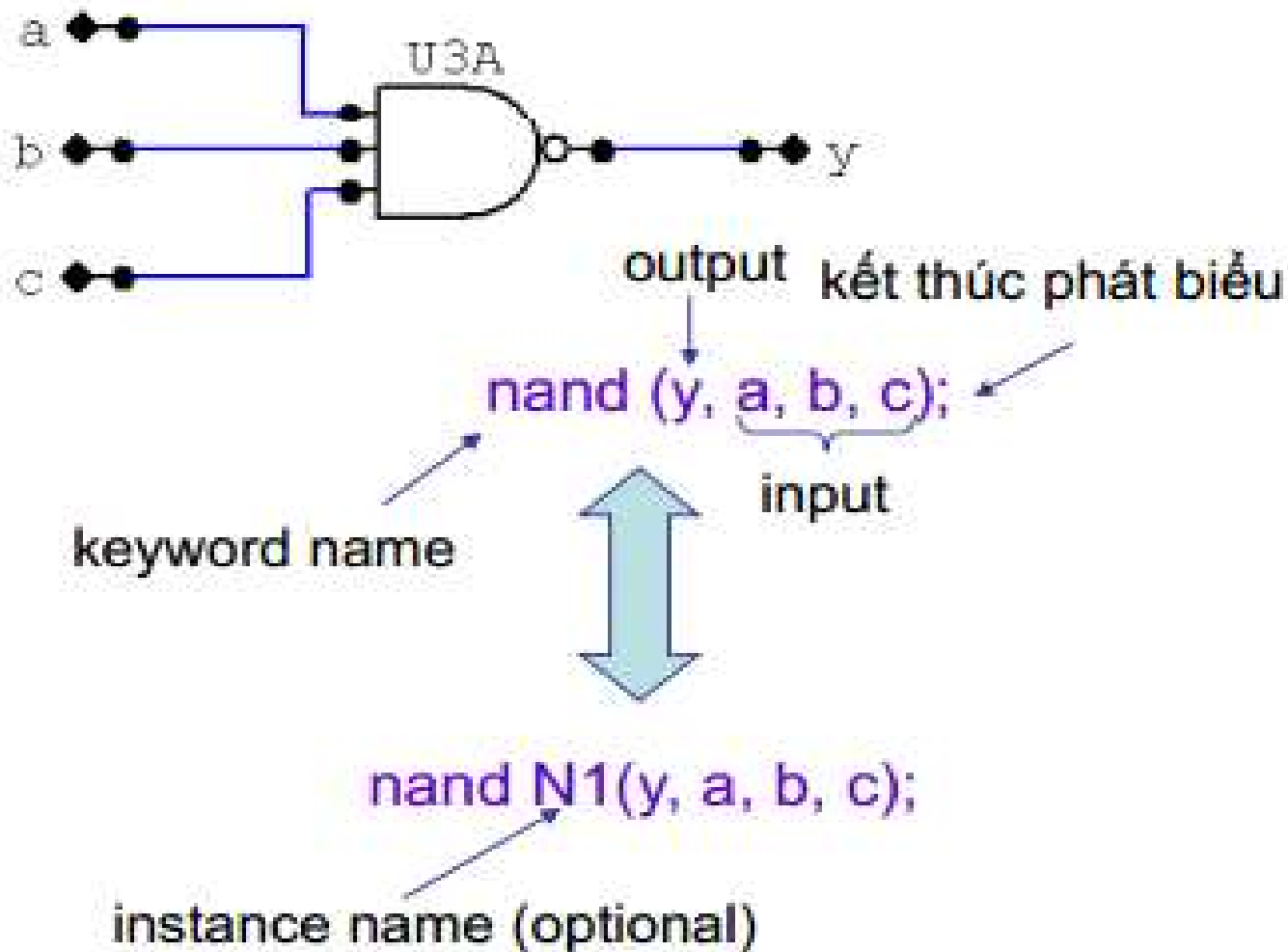
2. Các hàm logic được sử dụng trong Verilog

a. hàm and, nand, or, nor, xor, và xnor

Tên hàm: and, nand, or, nor, xor, xnor

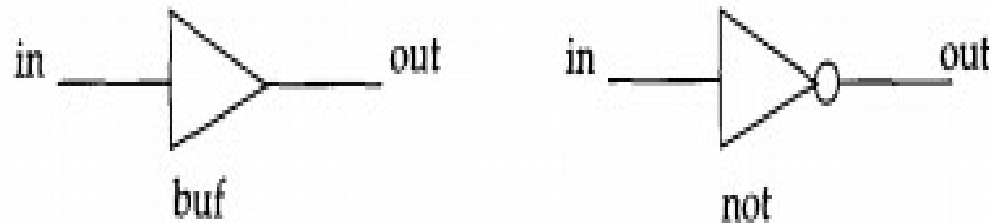


Ví dụ: Mô tả cổng NAND ba ngõ vào



b. Buf/Not Gates

Tên hàm: not, buf



Ví dụ: Mô tả cổng NOT

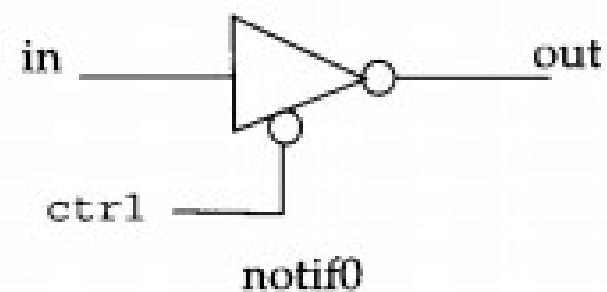
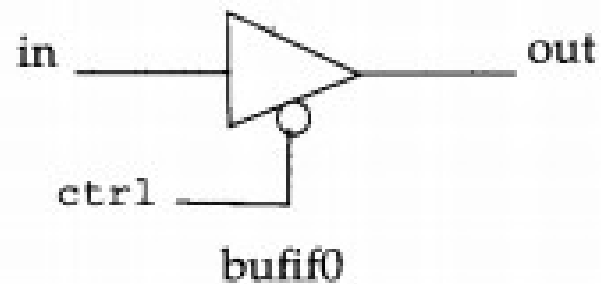
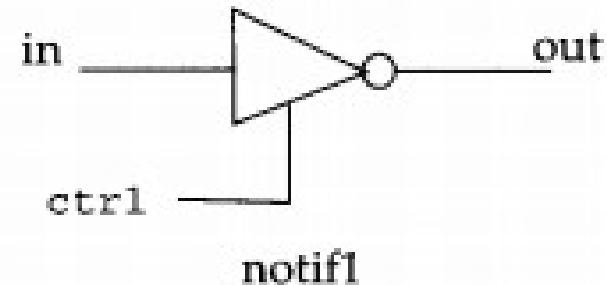
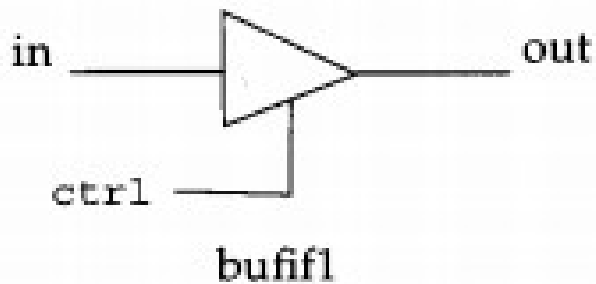
not (out,in)

hoặc not U(out,in) //U là tên cổng not



c. Bufif / notif

Tên hàm:	bufif1	notif1
	bufif0	notif0



Cách gọi hàm Bufif/notif

bufif1 (out,in,ctrl)

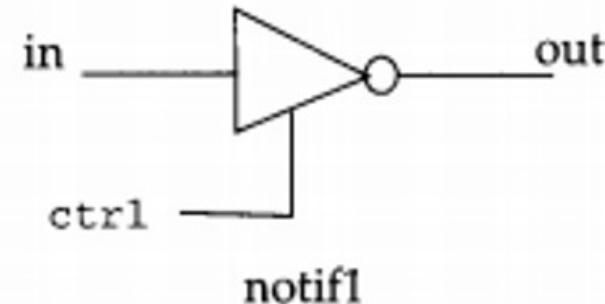
bufif0 (out,in,ctrl)

notif1 (out,in,ctrl)

notif0 (out,in,ctrl)

Ví dụ: Mô tả cổng not có chân cho phép tích cực cao

notif1 (out, in, ctrl)

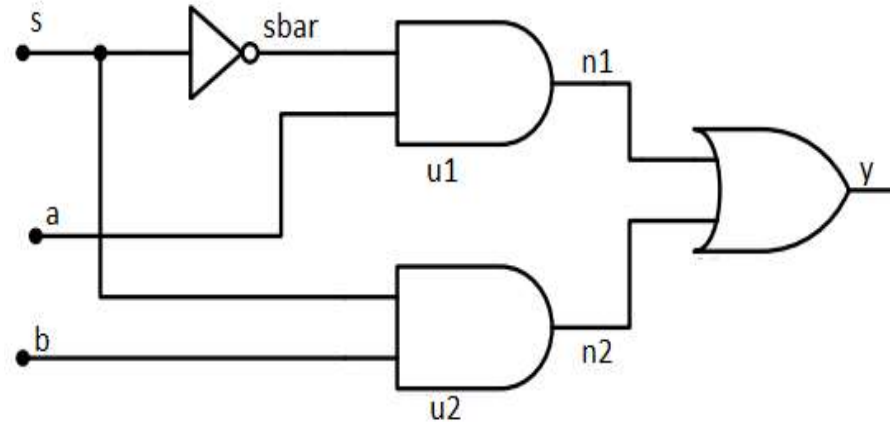


3. Các bước mô tả mạch tổ hợp dùng mô hình cấu trúc

- Xác định số ngõ vào/ ngõ ra của mạch tổ hợp
- Hiểu rõ nguyên lý hoạt động hoặc bảng trạng thái của mạch tổ hợp
- Viết biểu thức ngõ ra của mạch tổ hợp
- Vẽ mạch logic theo biểu thức ngõ ra
- Sử dụng các hàm có sẵn hoặc modul con để mô tả mạch tổ hợp



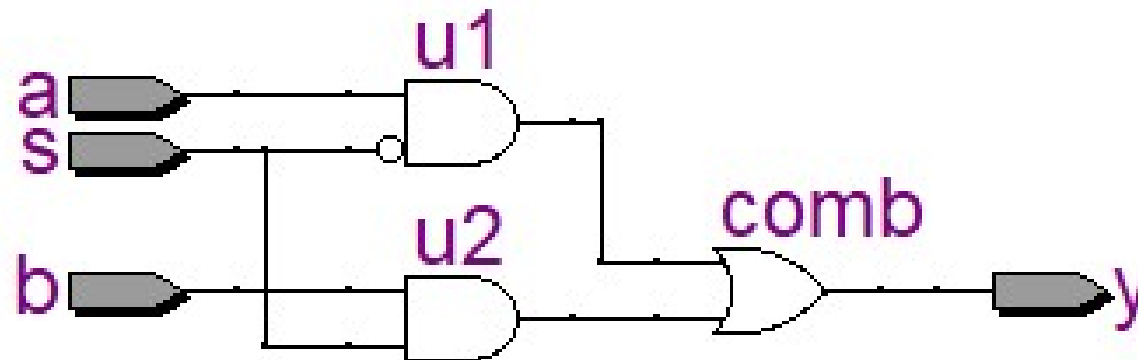
VD1 : mô tả mạch tổ hợp sau dùng mô hình cấu trúc



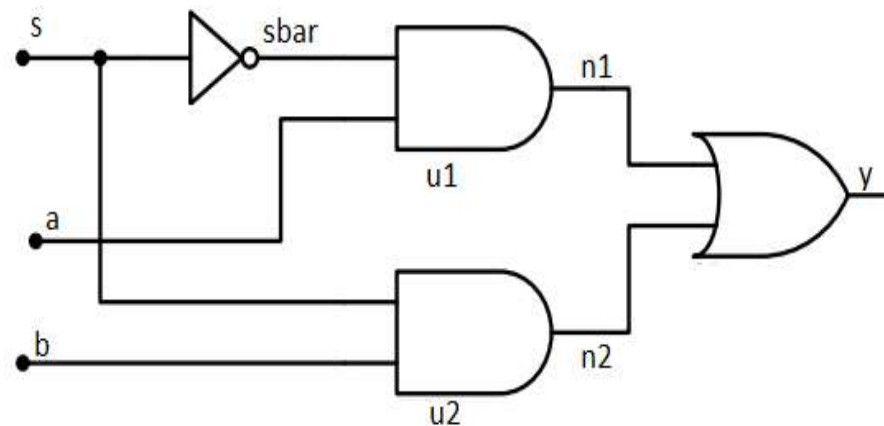
```
module machtohop (y, a, b, s);  
input a,b,s;  
output y;  
wire n1, n2, sbar ;  
    not (sbar, s);  
    and u1(n1, sbar, a);  
    and u2(n2, s, b);  
    or (y, n1, n2);  
endmodule
```



Kết quả chạy nestlist



Mạch logic ban đầu



VD 2: mô tả mạch cộng HA dùng mô hình cấu trúc

- Thiết kế số
- Vẽ mạch logic
- Viết code verilog

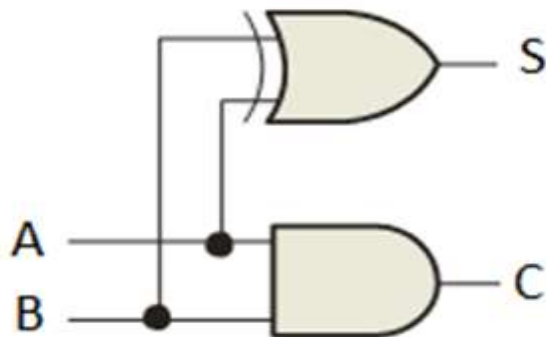
Bài giải:

Biểu thức ngõ ra:

$$S = A \oplus B$$

$$C = A . B$$

Sơ đồ mạch HA:



```
module HA (a, b, s,c);  
input a,b;  
output s, c;  
    and (c, a, b);  
    xor (s, a, b);  
endmodule
```



II. Mô hình luồng dữ liệu (DataflowModel)

1. Đặc điểm chung:

- Dataflow Model mô tả luồng dữ liệu bên trong hệ thống số giống như luồng dữ liệu giữa các thanh ghi
- Dataflow Model thường dùng để mô tả mạch tổ hợp thông qua các phép gán liên tục và biểu thức logic
- Dataflow Model thường dùng để mô tả các mạch số vừa và nhỏ



2. Phép gán liên tục

- Phép gán liên tục là phép gán sẽ được thực thi bất cứ khi nào giá trị của biểu thức bên phải thay đổi. Biểu thức bên trái phải có dữ liệu kiểu net.
- Cú pháp:

assign biến_wire = biểu thức;

ví dụ:

```
module and_gate(a,b,y);  
input a,b;  
output y;  
    assign y = a & b;  
endmodule
```



Các toán tử được sử dụng trong dataflow module

Operator Type	Operator Symbol	Operation Performed
Arithmetic	*, /, +, -	As expected
	%	Modulo
Logical	!	Logic NOT
	&&	Logic AND
		Logic OR
Bitwise	~	Bitwise NOT
	&	Bitwise AND
		Bitwise OR
	^	Bitwise XOR
	~^	Bitwise XNOR
Relational	<, >, <=, >=	As expected
Equality	==, !=	As expected
Reduction	&	Red. AND
	~&	Red. NAND
		Red. OR
	~	Red. NOR
	^	Red. XOR
	~^	Red. XNOR
Shift	<<	Left shift
	>>	Right shift
Concat	{ }	Concatenate
Replicate	{ { } }	Replicate
Cond	?:	As expected



Cấp độ ưu tiên của các toán tử trong dataflow module

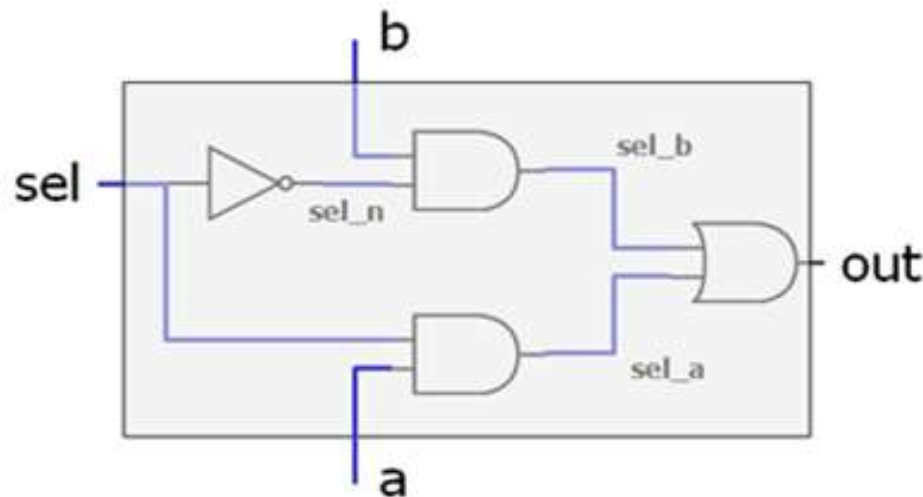
Xử lý logic	Toán tử	Mức ưu tiên
Logic negation and Bit-wise negation (Lấy bù)	!	Cao ↑ Thấp
Multiply, divide operations (Nhân, chia)	~	
	*	
	/	
Plus, minus operations (Cộng, trừ)	%	
	+	
Shift operation (Dịch)	-	
	<<	
Comparison operation (So sánh hơn)	>>	
	< <= > >=	
Logical Equality, case equality (sẽ sánh bằng)	< <= > >=	
Bit-wise AND operation	== != === !==	
Bit-wise exclusive operation	&	
Bit-wise OR operation	^ ^~	
Logical AND operation		
Logical OR operation	&&	
Conditional expression (Điều kiện)		
	?:	

3. Các bước mô tả mạch tổ hợp dùng mô hình luồng dữ liệu

- Xác định số ngõ vào/ ngõ ra của mạch tổ hợp
- Nắm rõ nguyên lý hoạt động hoặc bảng trạng thái của mạch tổ hợp
- Viết biểu thức ngõ ra của mạch tổ hợp
- Sử dụng phép gán assign hoặc modul con để mô tả mạch tổ hợp



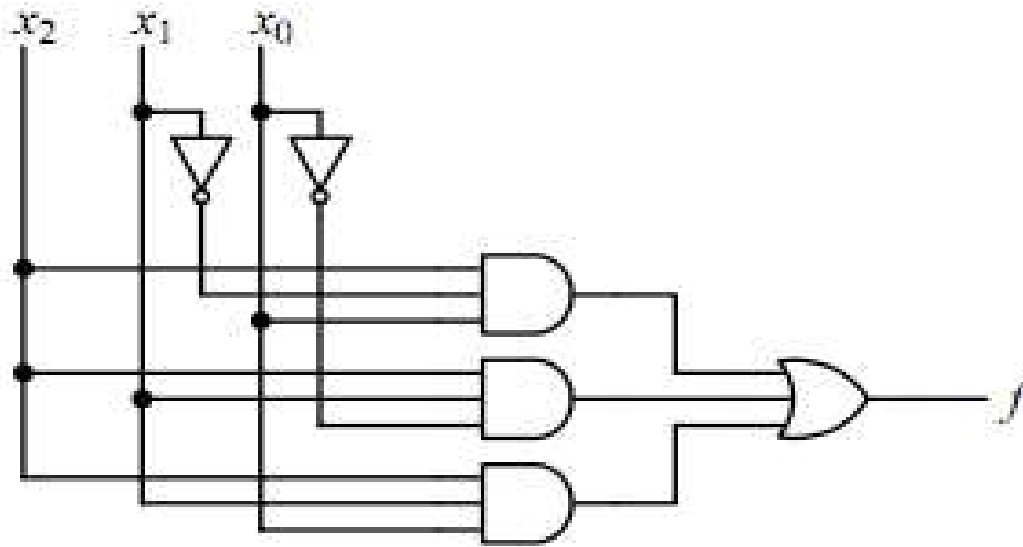
Ví dụ: mô tả mạch tổ hợp sau dung mô hình luồng dữ liệu



```
module machtohop (out, a, b, sel);  
  input  a, b, sel;  
  output out;  
  assign out = (sel & a) | (~sel & b);  
endmodule
```



Ví dụ: mô tả mạch tổ hợp sau dùng verilog dùng mô hình luồng dữ liệu và cấu trúc



$$f = x_2x_1'x_0 + x_2x_1x_0' + x_2x_1x_0$$

Cách 1:

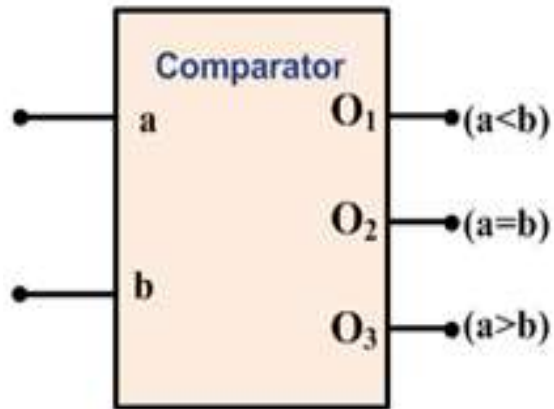
```
module vd2 (x0,x1,x2,f);
input x1,x2,x0;
output f;
wire n1,n2,n3,n4,n5;
not u1(n1,x1);
not u2(n2,x0);
and u3(n3,x2,n1,x0);
and u4(n4,x2,x1,n2);
and u5(n5,x2,x1,x0);
or u6(f,n3,n4,n5);
endmodule
```

Cách 2:

```
module vidu_df (x2,x1,x0,f);
input x1,x2,x0;
output f;
assign f = (x2&(~x1)&x0)|(x2&(x1)&(~x0))|(x2&x1&x0);
endmodule
```



Ví dụ: mô tả mạch so sánh 1 bit dùng dataflow model

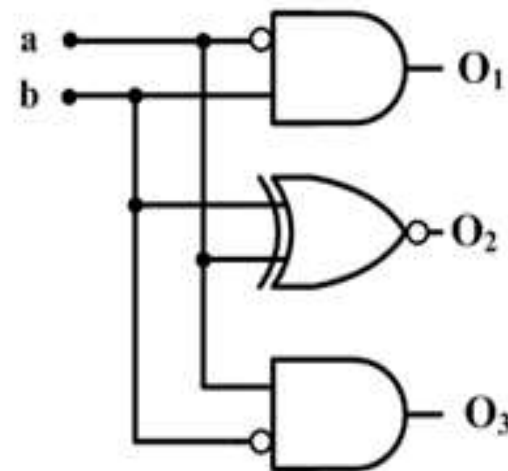


Input		Output		
a	b	O ₁	O ₂	O ₃
0	0	0	1	0
0	1	1	0	0
1	0	0	0	1
1	1	0	1	0

$$O_1 = \bar{a}b$$

$$O_2 = \overline{a \oplus b}$$

$$O_3 = a\bar{b}$$



Ví dụ: mô tả mạch so sánh 1 bit dùng dataflow model

```
module compare1bit (o1,o2,o3,a,b);  
  input a,b;  
  output o1, o2, o3;  
  
  assign o1 = ~a & b;  
  assign o2 = a ~^ b;  
  assign o3 = a & ~b;  
  
endmodule
```



Ví dụ: mô tả mạch cộng 4 bit dùng dataflow model

```
module fulladd4(sum, c_out, a, b, c_in);  
    output [3:0] sum;  
    output c_out;  
    input[3:0] a, b;  
    input c_in;  
    assign {c_out, sum} = a + b + c_in;  
endmodule
```



III. Mô hình hành vi (Behavioral model)

1. Đặc điểm chung:

- Mô hình hành vi mô tả hệ thống theo cách mà nó hành xử thay vì kết nối các linh kiện ở mức thấp lại với nhau
- Mô hình hành vi mô tả mối quan hệ giữa các tín hiệu ngõ ra với các tín hiệu ngõ vào mà không cần quan tâm đến cấu trúc phần cứng bên trong nó
- Mức trừu tượng cao hơn và thường để mô tả các mạch số lớn



2. Các bước mô tả mạch tổ hợp dùng mô hình hành vi

- Xác định số ngõ vào/ ngõ ra của mạch tổ hợp
- Nắm rõ nguyên lý hoạt động hoặc bảng trạng thái của mạch tổ hợp
- Sử dụng cấu trúc always, if, case, ... để mô tả hoạt động của mạch tổ hợp
- Trong một modul có thể dùng một hoặc nhiều khối always

Lưu ý khi dùng always:

- Các biến nhảy thường là ngõ vào của mạch tổ hợp
- Các biến phải khai báo dạng reg



Ví dụ : Mô tả decoder 2 to 4, ngõ ra tích cực cao, chân cho phép (e) tích cực cao dùng mô hình hành vi



```
/* mô tả decoder2_4 dùng  
phát biểu case */
```

```
module decoder2_4 (y,x,e);  
input [1:0]x;  
input e;  
output [3:0] y;  
reg [3:0] y;
```

```
always @ (x or e)  
begin  
    if (e == 1'b1)  
        y = 4'b0000;  
    else  
        begin  
            case (x)  
                2'b00: y = 4'b0001;  
                2'b01: y = 4'b0010;  
                2'b10: y = 4'b0100;  
                2'b11: y = 4'b1000;  
            endcase  
        end  
    end  
end  
endmodule
```



VD: Mô tả ALU 3 bit với 8 chức năng

s_2	s_1	s_0	Operation Name	Operation
0	0	0	Pass	Pass A to output
0	0	1	AND	$A \text{ AND } B$
0	1	0	OR	$A \text{ OR } B$
0	1	1	NOT	A'
1	0	0	Addition	$A + B$
1	0	1	Subtraction	$A - B$
1	1	0	Increment	$A + 1$
1	1	1	Decrement	$A - 1$



```
module alu_bh (a, b,s, f);  
input [2:0]a, b,s;  
output [3:0]f;  
reg [3:0]f;  
always @ (a, b, s) begin  
    case (s)  
        3'b000 : f = a;  
        3'b001 : f = a&b;  
        3'b010 : f = a|b;  
        3'b011 : f = ~a;  
        3'b100 : f = a+b;  
        3'b101 : f = a-b;  
        3'b110 : f = a + 1'b1;  
        3'b111 : f = a - 1'b1;  
    endcase  
end  
endmodule
```



IV. Mô tả mạch tổ hợp dùng cấu trúc phân cấp

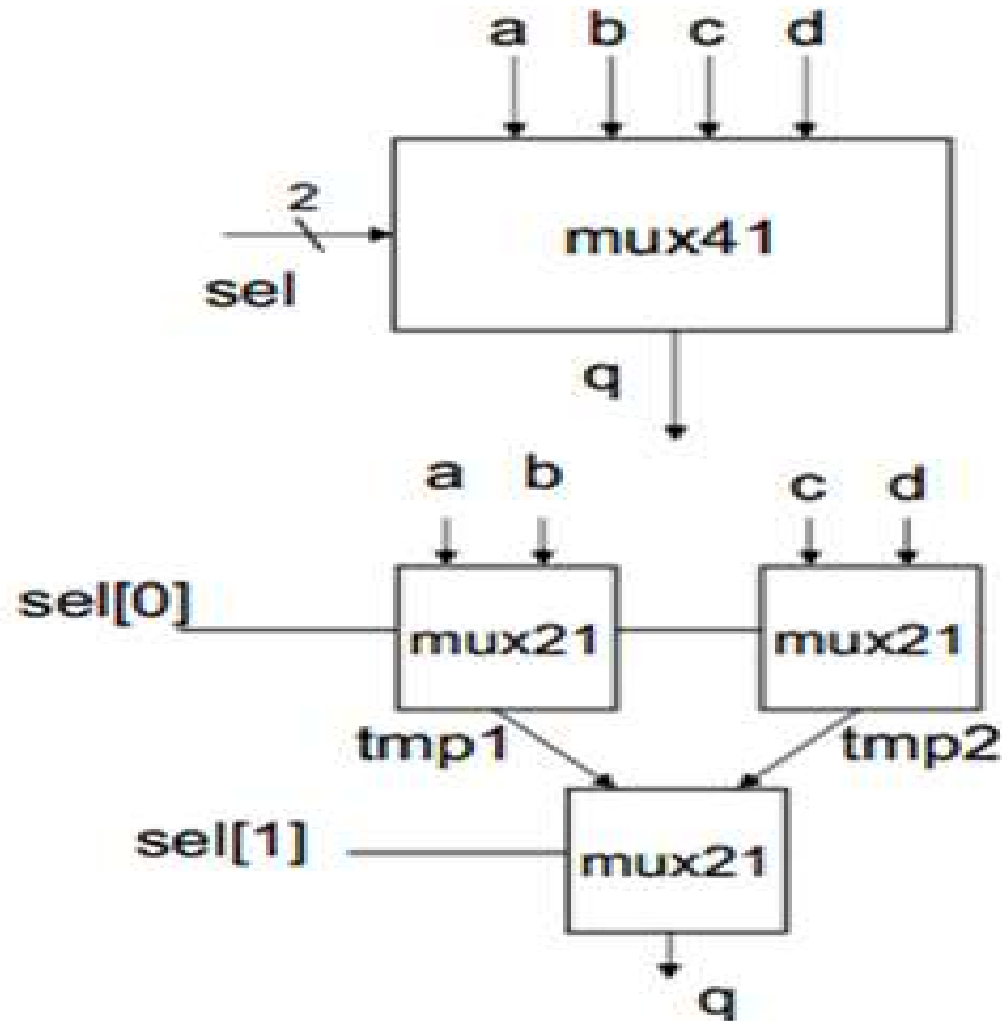
Các bước thực hiện:

- Xác định số ngõ vào/ ngõ ra của mạch tổ hợp
- Hiểu rõ nguyên lý hoạt động hoặc bảng trạng thái của mạch tổ hợp
- Vẽ sơ đồ logic theo cấu trúc phân cấp
- Viết code mô tả module top, trong đó sử dụng các hàm gọi modul con theo sơ đồ mạch logic
- Viết code mô tả các module con

Lưu ý: các modul con mô tả mạch tổ hợp cấp thấp hơn modul top và có thể viết cùng hoặc khác file với module top

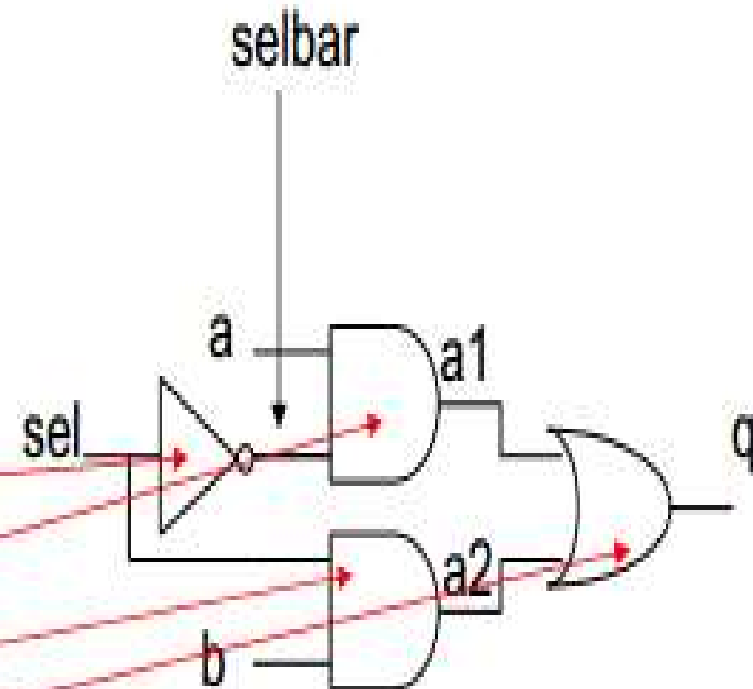


VD: Mô tả Mux 4 to 1 sử dụng Mux 2 to 1



VD: Mô tả Mux 2 to 1

```
module mux21(q, sel, a, b);  
  input sel, a, b;  
  output q;  
  wire selbar, a1, a2;  
  
  not(selbar, sel);  
  and(a1, selbar, a);  
  and(a2, sel, b);  
  or(q, a1, a2);  
  
endmodule
```



VD: mô tả MUX 4 to 1 dùng MUX 2to1

//module top

```
module mux4to1(q,sel,a,b,c,d);  
  input [1:0] sel;  
  input a,b,c,d;  
  output q;  
  wire tmp1,tmp2;  
  
  mux2to1 M0 (tmp1,sel[0],a,b);  
  mux2to1 M1 (tmp2,sel[0],c,d);  
  mux2to1 M2 (q,sel[1],tmp1,tmp2);  
endmodule
```

//module instantiation

```
module mux2to1(q,s,a,b);  
  input s,a,b;  
  output q;  
  wire selbar,a1,a2;  
  
  not(selbar,s);  
  and (a1,selbar,a);  
  and (a2,s,b);  
  or (q,a1,a2);  
endmodule
```





Thank you for
listening

